

Assignment: Functions and Parameter Passing in C

1. Function: getUserInitials

```
#include <stdio.h>

#include <ctype.h>

void getUserInitials(char name[])
{
    printf("Initials: %c%c\n", toupper(name[0]), toupper(name[6]));
}

int main()
{
    getUserInitials("Virat Kohli");
    return 0;
}
```

Output:

Initials: VK

2. Function: addToCart (Pass by Reference Using Arrays)

```
#include <stdio.h>

#include <string.h>

void addToCart(char cart[][30], int *count, char product[])
{
    strcpy(cart[*count], product);
    (*count)++;
}
```

```

    printf("Updated Cart:\n");
    for(int i = 0; i < *count; i++)
    {
        printf("%s\n", cart[i]);
    }
}

int main()
{
    char cart[10][30] = {"Laptop", "Mouse"};
    int count = 2;

    addToCart(cart, &count, "Keyboard");

    printf("\nCart Outside Function:\n");
    for(int i = 0; i < count; i++)
    {
        printf("%s\n", cart[i]);
    }

    return 0;
}

```

Output:

Updated Cart:

Laptop

Mouse

Keyboard

Cart Outside Function:

Laptop

Mouse

Keyboard

This demonstrates that changes made through reference persist outside the function.

3. Pass-by-Value vs Pass-by-Reference

```
#include <stdio.h>
```

```
void increaseFollowersByValue(int followers)
```

```
{  
    followers += 1000;  
    printf("Inside Pass-by-Value: %d\n", followers);  
}
```

```
void increaseFollowersByReference(int *followers)
```

```
{  
    *followers += 1000;  
    printf("Inside Pass-by-Reference: %d\n", *followers);  
}
```

```
int main()
```

```
{
```

```
int followers = 5000;

increaseFollowersByValue(followers);

printf("After Pass-by-Value: %d\n\n", followers);

increaseFollowersByReference(&followers);

printf("After Pass-by-Reference: %d\n", followers);

return 0;
}
```

Output:

Inside Pass-by-Value: 6000

After Pass-by-Value: 5000

Inside Pass-by-Reference: 6000

After Pass-by-Reference: 6000

4. Function: formatPrice

```
#include <stdio.h>
```

```
void formatPrice(int price)
```

```
{
    printf("₹%d\n", price);
}
```

```
int main()
```

```
{  
    printf("Product Prices:\n");  
  
    formatPrice(1599);  
    formatPrice(24999);  
    formatPrice(799);  
  
    return 0;  
}
```

Output:

Product Prices:

₹1599

₹24999

₹799

5. Reusable Function: Capitalize First Letter

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
void capitalizeFirstLetter(char str[])
```

```
{  
    str[0] = toupper(str[0]);  
    printf("%s\n", str);  
}
```

```
int main()
```

```
{  
    char product[] = "laptop";  
    char username[] = "virat18";  
  
    printf("Product Name: ");  
    capitalizeFirstLetter(product);  
  
    printf("Username: ");  
    capitalizeFirstLetter(username);  
  
    return 0;  
}
```

Output:

Product Name: Laptop

Username: Virat18

This function is reusable because it works with any string input, including product names, usernames, and other text.

Conclusion

This assignment demonstrated the use of functions, parameter passing, pass-by-value, pass-by-reference, array manipulation, string formatting, and reusable function design in C programming.